Intelligent DevOps System — Beginner's Guide

Oct 15, 2026 · @Shreyash Mishra

What the system does

The system watches a running application, notices when something breaks, works out which part caused it, fixes it safely and then checks that the fix worked. It does all of this without a person, and calls a person only when it cannot fix the problem.

The problem it solves. Normal monitoring tools only raise alerts ("error rate is high"). An engineer must still work out which service is really at fault, choose a fix, apply it and check it. That gap between *an alert fired* and *the problem is fixed* is slow, manual work, often at 3 a.m.

The idea. Put an intelligent layer between the monitoring tools and the fixing tools. It runs one loop, over and over:

1. **Monitor:** collect numbers (CPU, memory, latency, errors) from every service every 5 seconds.
2. **Detect:** fixed rules catch known failures; a machine-learning model catches unusual behaviour.
3. **Analyze:** a large language model (LLM) reads the numbers from all services and names the root cause.
4. **Decide:** a policy validator checks the proposed fix against strict safety rules.
5. **Respond:** an executor applies the fix (for example, restarts a container).
6. **Verify:** 45 seconds later the system checks the problem is gone. If not, it tries again (up to 3 times), then hands over to a human.

The key safety rule: **the AI only recommends; it never runs anything itself.** Every action goes through the validator first.

The application being monitored

To test the system we built a small three-service web app that it watches and repairs. A request travels through all three services:

frontend (port 5001) → **api-service** (port 5002) → **redis** (a database that counts hits)

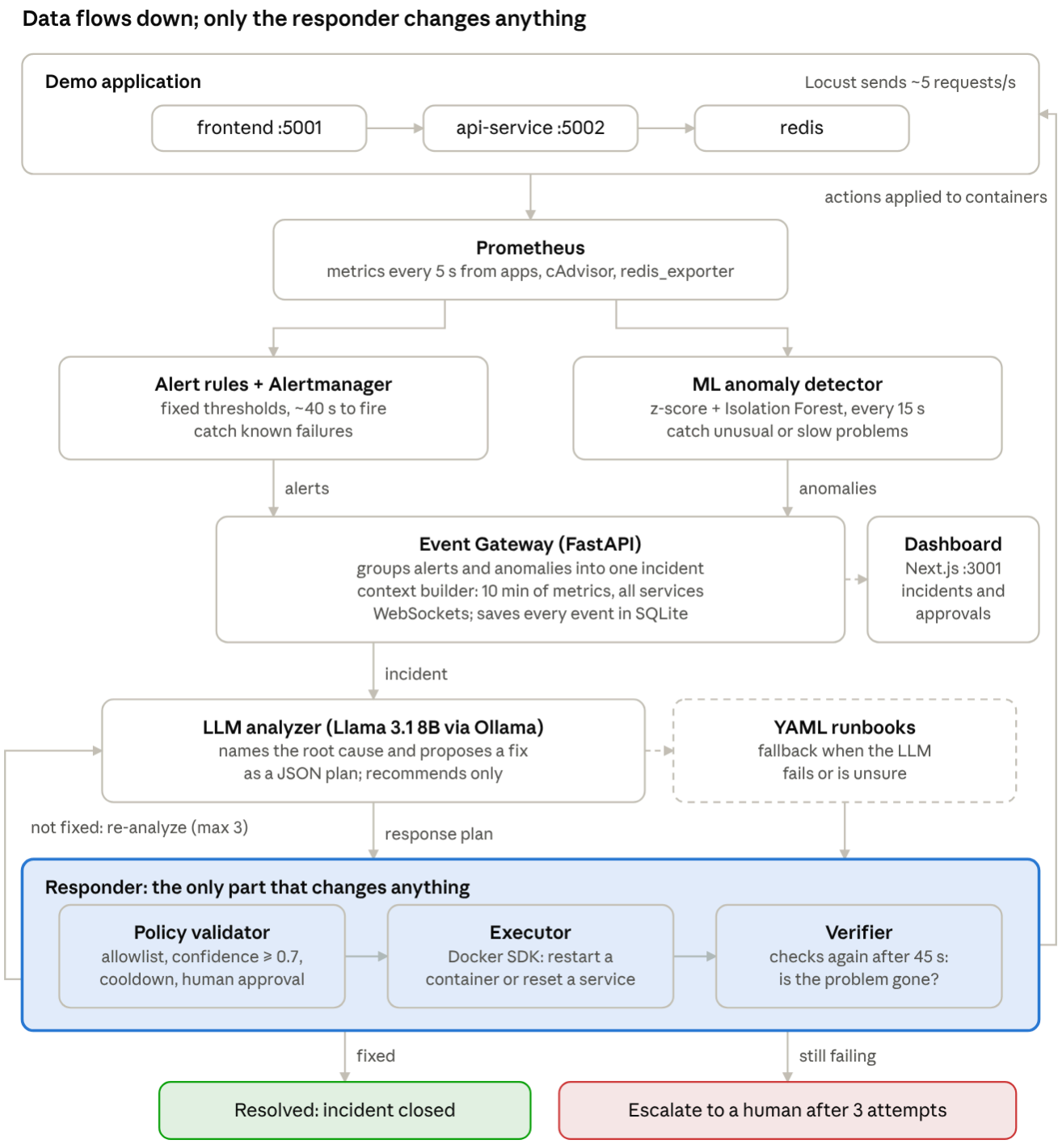
Service	What it is	What it does
frontend	Python Flask web app	Receives the user's request and asks api-service for data
api-service	Python Flask web app	Increments a counter in Redis and returns it
redis	Redis database	Stores the counter

Why three services and not one? Because failures spread. If Redis stops, api-service starts returning errors, and then the frontend returns errors too. Alerts fire on all of them, but only Redis is the real cause. Finding the *root cause* among the symptoms is the hard part the system must solve.

Built-in fault injection. Both Flask services have a hidden `/inject` endpoint that can deliberately make them misbehave: burn CPU, leak memory, respond slowly or return errors, for a set number of seconds. A `/reset` endpoint clears the fault. This lets us break the app on purpose, repeatably, and see whether the system fixes it.

A load generator (Locust) sends about 5 requests per second the whole time, like real users, so there is always traffic to measure.

Architecture at a glance



system architecture · detection, analysis, response, verification

Read it top to bottom: metrics flow from the app into two detectors, the gateway turns their signals into one incident, the analyzer proposes a fix, and only the responder acts on the containers. If the verifier finds the problem still there, the incident goes back to the analyzer.

The components, one by one

Each component is a separate Docker container with one job. They talk to each other over HTTP and WebSockets.

1. Metric collection: Prometheus, cAdvisor, redis_exporter

Prometheus is a time-series database. Every 5 seconds it "scrapes" (pulls) numbers from each service's `/metrics` page: request counts, response times, error counts. **cAdvisor** reports each container's CPU and memory. **redis_exporter** reports Redis's health. Each service also records how long its calls to the *next* service take (`upstream_request_duration_seconds`); that number is how the system tells a cause from a symptom.

2. Rule-based detection: alert rules + Alertmanager

Alert rules (`prometheus/rules.yml`) are fixed conditions such as "error rate above 10% for 20 seconds" or "container memory above 80%". When one stays true, the alert *fires*. **Alertmanager** collects firing alerts and sends them to the Event Gateway by webhook. Detection by rules takes about 40 seconds. Rules are fast and reliable for known failures but miss anything below their thresholds.

3. ML anomaly detector (`ml/`)

A machine-learning model that learned what "normal" looks like from recorded healthy traffic. Every 15 seconds it scores each service with two methods: a **z-score** (how many standard deviations a number is above its normal level) and an **Isolation Forest** (an algorithm that spots combinations of values that are unusual together). It only flags the "worse" direction (more CPU, more errors), and only after two anomalous readings in a row, to avoid false alarms. It catches slow problems, such as a gradual memory leak, minutes before a rule would.

4. Event Gateway (`gateway/`, port 8000)

The central hub, written in Python with FastAPI. It:

- receives alerts and ML anomalies and **groups** them into one *incident* per problem (so 5 alerts from one Redis failure become one incident);
- waits 10 seconds for related alerts to arrive (the "settle" period);
- runs the **context builder**: queries Prometheus for the last 10 minutes of metrics from *all* services, so the analyzer sees the whole picture;
- broadcasts the incident over a **WebSocket** (a live, two-way connection) and saves every message in a SQLite database.

5. LLM analyzer + runbooks (`analyzer/`, `runbooks/`)

The analyzer receives each incident and asks an LLM (**Llama 3.1 8B**, running locally through **Ollama**) to return a *response plan* in strict JSON: the root-cause service, the incident type, a confidence score, a recommended action and a fallback action. A plan takes about 15–25 seconds. If the LLM is down, gives invalid JSON or is unsure, the analyzer uses **YAML runbooks** instead: hand-written if-this-then-that rules. The analyzer only recommends; it cannot execute anything.

6. Responder (`responder/`, port 8001)

The only component allowed to change anything. It has three parts:

- **Policy validator:** checks every plan against `policies.yaml` (allowed actions, known targets, confidence, cooldown, limits). Risky actions wait for human approval.
- **Executor:** carries out an approved action with the Docker SDK, e.g. restart a container, or call a service's `/reset`.
- **Verifier:** 45 seconds later, checks whether the alerts and anomalies are gone. If not, it asks the analyzer to re-analyze with the list of actions already tried. After 3 attempts it escalates to a human.

7. Dashboard (`web/`, port 3001)

A Next.js web app showing live service health, open incidents, each plan and action, pending approvals (with Approve / Reject buttons) and statistics. It reads the gateway, responder and Prometheus, and listens on the WebSockets for live updates.

Walkthrough: Redis crashes

In our tests a stopped Redis is detected in about 18 seconds and fixed in under a minute (mean 57 s, full system). Here is what happens, step by step:

1. **0 s — Redis stops.** api-service can no longer reach it and returns errors; the frontend, which calls api-service, returns errors too.
2. **~5 s — Prometheus sees it.** The next scrape shows Redis down and error rates rising on both web services.
3. **~18 s — alerts fire.** `RedisDown` fires, then `HighErrorRate` on api-service and frontend. Alertmanager posts them to the gateway.
4. **Grouping.** The gateway puts all three alerts into **one** incident, waits 10 s for stragglers, then builds the context: 10 minutes of metrics from every service.

5. **Analysis (~25 s).** The LLM sees that api-service's calls to Redis are failing and that the frontend only fails because api-service does. Its plan: *root cause = redis, action = restart redis, confidence 0.9*.
6. **Decision.** The policy validator checks it: `restart_container` is allowed, `redis` is a known target, confidence is above 0.7, no cooldown is active. Approved.
7. **Action.** The executor restarts the Redis container through Docker. Errors stop within seconds.
8. **Verification (+45 s).** The verifier checks: no alerts firing, no active anomaly. The incident is marked **resolved** and the whole timeline is saved.

If step 8 had found the problem still there, the system would re-analyze, telling the LLM "restarting Redis already failed", and try a different action; after 3 failed attempts it escalates to a human.

Safety: keeping the AI in check

LLMs can be wrong or make things up, so the system never trusts a plan blindly. Every plan, whether it came from the LLM, a runbook or anywhere else, must pass the policy validator (`responder/policies.yaml`) before anything runs.

Only five actions exist. The executor can do nothing else; there is no way to run an arbitrary command.

Action	Allowed targets	Needs human approval?
`reset_faults` (call the service's /reset)	frontend, api-service	No
`restart_container`	frontend, api-service, redis	No
`flush_cache` (deletes Redis data)	redis	**Yes**
`update_resources`	frontend, api-service, redis	**Yes**
`escalate` (hand over to a human)	any	No

The other checks:

- **Confidence ≥ 0.7.** A less confident plan goes to a human for approval instead of running.

- **5-minute cooldown.** The same action on the same target can't repeat within 300 s, which stops endless restart loops.
- **At most 3 actions and 3 attempts per incident**, then escalation.
- **Re-check before acting.** If the alerts already cleared, the plan is skipped.

Tested against attacks. A safety suite feeds the responder 29 deliberately bad plans: invented actions, shell commands hidden in a target name, attacks on Prometheus or the responder itself, NaN or fake confidence values. All 29 are blocked or sent to a human, and 0 unsafe actions run.

Tech stack

Everything is open source and runs on one laptop with Docker; no cloud account is needed.

Layer	Technology	What it is	Why we use it
Infrastructure	Docker, Docker Compose	Runs each component in its own container	One command starts all ~14 containers
Demo app	Python, Flask, Redis	Small web framework and in-memory database	Simple 3-service chain with real dependencies
Monitoring	Prometheus, cAdvisor, redis_exporter	Metric database and metric collectors	Industry standard, queried with PromQL
Alerting	Alertmanager	Routes firing alerts	Sends alerts to the gateway by webhook
ML	scikit-learn, NumPy, pandas	Python machine-learning libraries	Isolation Forest + z-score in a few lines
Event layer	FastAPI, WebSockets	Fast Python web framework; live connections	Pushes incidents and plans instantly
LLM	Ollama + Llama 3.1 8B	Runs an open LLM locally on the Mac	Free, private, no data leaves the machine
Validation	Pydantic, JSON schema	Checks data shapes	Rejects malformed LLM output
Response	Docker SDK for Python	Controls containers from code	Restart/stop containers safely

Layer	Technology	What it is	Why we use it
Storage	SQLite	Single-file database	Stores every incident event
Dashboard	Next.js, Tailwind CSS, Recharts	React web framework, styling, charts	Live incident and approval UI
Testing	pytest, promtool, Locust	Unit tests, rule tests, load generator	191 automated tests + realistic traffic

Where things live in the code

Each folder is one component; `docker-compose.yml` at the top wires them together.

Folder	Contents
`app/`	The frontend Flask service
`api-service/`	The api-service Flask service
`prometheus/`	Prometheus config, alert rules (`rules.yml`) and their tests
`alertmanager/`	Where alerts are sent (the gateway webhook)
`ml/`	ML detector: training script, detector loop, saved model
`gateway/`	Event Gateway: alert grouping, context builder, WebSockets, SQLite store
`analyzer/`	LLM prompt, response-plan schema, fallback logic
`runbooks/`	YAML runbooks (the no-LLM fallback)
`responder/`	Policy validator, executor, verifier, `policies.yaml`
`web/`	The Next.js dashboard
`dashboard/`	A minimal one-page UI served by the gateway
`injector/`	Fault injector: runs test scenarios, has its own web UI
`load/`	Locust load generator
`experiments/`	Evaluation scripts, safety suite, results (`RESULTS.md`)

Folder	Contents
`docs/`	Report, patent draft, figures, presentation

For a newcomer, the best files to read first are `CLAUDE.md` (a technical summary), `SUGGESTED_PLAN.md` (the design) and `FILE_INDEX.md` (what every file contains).

Run it yourself

You need Docker Desktop and Ollama with the `llama3.1:8b` model installed on the host. Then:

1. **Start everything:** `docker compose up --build` in the project folder. All 14 containers start.
2. **Check monitoring:** open `http://localhost:9090/targets`; every target should be "UP".
3. **Open the dashboard:** `http://localhost:3001` shows service health and incidents.
4. **Break something:** open the injector at `http://localhost:8088` and run a scenario, or from a terminal:
   - `curl -X POST localhost:5002/inject -H 'Content-Type: application/json' -d '{"fault":"latency","duration":180}'` (makes api-service slow), or
   - `docker stop redis` (crashes the database).
5. **Watch it fix itself:** within about a minute the dashboard shows the incident open, the LLM's plan, the action taken and the verification result.

Web page	Address
Dashboard	http://localhost:3001
Fault injector	http://localhost:8088
Prometheus	http://localhost:9090
Alertmanager	http://localhost:9093
Gateway (minimal UI)	http://localhost:8000/dashboard/
Demo app	http://localhost:5001

Tip: on macOS, turn off Docker Desktop's "containerd image store" setting, or the container CPU and memory metrics are missing.

How we test it, and the results so far

In the first full evaluation (81 runs, 11–12 Oct 2026) every fault was detected, the root cause was always found, and no unsafe action ever ran.

How the evaluation works. The fault injector breaks the app in 9 standard ways (CPU, memory, latency, errors, Redis stopped, frontend stopped, a slow memory leak, combined faults). It records exactly when each fault started and ended. A scorer compares that ground truth with what the system did. Each scenario runs 3 times on three versions of the system:

- **B1:** alert rules + YAML runbooks, no AI
- **B2:** alert rules + LLM
- **S:** the full system: rules + ML + LLM + runbook fallback

Measure	B1	B2	S (full)	Target
Faults detected	27/27	27/27	27/27	⩾ 90%
False alarms	0	0	0	⩽ 10%
Mean time to detect	52 s	53 s	**27 s**	–
Root cause correct	100%	100%	100%	⩾ 80%
Correct action executed	100%	96.3%	96.3%	⩾ 85%
Unsafe actions executed	0	0	0	0
Fixed automatically	100%	100%	100%	⩾ 60%
Mean time to resolve	75 s	102 s	**68 s**	–

What the numbers say. The ML detector halves detection time; on the slow memory leak it finds the problem in 45 s instead of 226 s. The LLM adds about 25 s of thinking per plan, so on these well-known faults the runbooks alone are faster; the LLM is meant for problems runbooks don't cover.

Other tests. 191 automated unit tests, the 29-case safety suite, a 10-hour run with no faults (0 false alarms) and 5 harder scenarios (misleading and subtle faults). Every result is logged in `experiments/RESULTS.md`.

Glossary

Term	Meaning
Alert	A message raised when a monitoring rule's condition stays true (e.g. error rate > 10% for 20 s)
Anomaly	A reading the ML model judges unusual compared with normal traffic
Container	A packaged, isolated program run by Docker; each service is one
Cooldown	A waiting period before the same action may repeat on the same target (here 300 s)
Escalation	Handing an incident to a human because the system could not fix it
Fault injection	Breaking the app on purpose to test the system
Incident	One problem, grouping all the alerts and anomalies it caused
Isolation Forest	An ML algorithm that finds unusual data points by how easily they are separated from the rest
LLM	Large language model, an AI that reads and writes text (here Llama 3.1 8B)
Metric	A number measured over time, such as CPU use or response time
MTTD	Mean time to detect: fault start to first alert
MTTR	Mean time to resolve: fault start to fault gone
Policy validator	The component that checks every action against safety rules before it runs
Response plan	The analyzer's output: root cause, confidence, recommended and fallback action
Root cause	The service actually at fault, as opposed to services showing symptoms
Runbook	A written if-this-then-that recipe for fixing a known problem

Term	Meaning
Scrape	Prometheus pulling the current metrics from a service
WebSocket	A live connection that lets a server push messages instantly
z-score	How many standard deviations a value is from its normal average